Phases of a Compiler

Presented By
Deva Dharshini(192472350)
Course Code: CSA1410
Course: Compiler Design

Guided By:
Dr. N Deepa

How a compiler works

Each phase reads the output of the previous one and passes its own result forward.

01

Lexical Analysis

Output: tokens

02

Syntax Analysis

Output: parse tree

03

Semantic Analysis

Output: annotated tree

04

Intermediate Code

Output: three-address code

05

Code Optimization

Output: improved code

06

Code Generation

Output: machine code

PHASE 1 · SCANNER

Lexical Analysis

The scanner reads the source code character by character and groups the characters into tokens.

- Removes whitespace and comments
- Matches patterns using regular expressions
- Reports invalid symbols as lexical errors

Output: stream of tokens

SOURCE CODE

position = initial + rate * 60

TOKENS

id(position) = id(initial)
+ id(rate) * num(60)

PHASE 2 · PARSER

Syntax Analysis

The parser checks that the tokens follow the grammar of the language and builds a parse tree.

- Uses a context-free grammar
- Reports errors such as a missing bracket
- Common tools: Yacc and Bison

Output: parse tree



PHASE 3 · MEANING CHECK

Semantic Analysis

The analyzer checks that the program makes sense, using the tree and the symbol table.

- Type checking of operands
- Variables must be declared before use
- Inserts type conversions where needed

Output: annotated tree

TYPES FOUND

rate : float

60 : int

AFTER CONVERSION

rate * inttofloat(60)

PHASE 4 · MIDDLE STEP

Intermediate Code Generation

The compiler turns the tree into a simple, machine-independent form between source and target code.

- Commonly three-address code
- At most one operator per instruction
- Easy to optimize and to translate

Output: three-address code

THREE-ADDRESS CODE

```
t1 = inttofloat(60)
t2 = rate * t1
t3 = initial + t2
position = t3
```

PHASE 5 · IMPROVE

Code Optimization

The optimizer makes the code faster and smaller without changing what the program does.

- Constant folding: 60 becomes 60.0 at compile time
- Removes unused temporaries and dead code
- Improves loops and repeated calculations

Output: improved intermediate code

BEFORE

```
t1 = inttofloat(60)
t2 = rate * t1
t3 = initial + t2
position = t3
```

AFTER

```
t1 = rate * 60.0
position = initial + t1
```

PHASE 6 · TARGET

Code Generation

The final phase translates the optimized code into instructions for the target machine.

- Selects machine instructions
- Allocates registers to values
- Orders instructions for the processor

Output: target machine code

ASSEMBLY CODE

```
LDF R2, rate
MULF R2, R2, #60.0
LDF R1, initial
ADDF R1, R1, R2
STF position, R1
```

Conclusion

- Compiler lets us to write programs in human-friendly languages and still run them on machines.
- Breaking compilation into phases keeps each step simple, testable, and reusable.
- The front end (lexical, syntax, semantic) catches errors early, before the program ever runs.
- The back end (intermediate code, optimization, code generation) makes the program fast and efficient.
- Understanding these phases is the foundation for building compilers, interpreters, and programming tools.


Phases of a Compiler
Presented By
Deva Dharshini
Course Code: CSA1410
Course: Compiler Design
205
GPS Map Camera
Mevalurkuppam, Tamil Nadu, India
22g8+cwh, Mevalurkuppam, Tamil Nadu 602105, India
Lat 13.026375° Long 80.017219°
Tuesday, 13/10/2026 02:05 PM GMT +05:30
Google